CppHDL Specification

Mike Reznikov

2026-08-21

This document is currently in **draft** status.
Content may change significantly before final approval.

Contents

Mapping of SystemVerilog Expressions to C++

CppHDL code should be read as a direct C++ mapping of synthesizable SystemVerilog RTL. Continuous assignments and module port connections are declared in port member initializers or written in the `_assign()` section. This connection setup runs only once, before the work cycle starts, and binds C++ lambdas that are used later during simulation and SystemVerilog generation. The `_ASSIGNxxx()` macros are only allowed in those static connection contexts.

SystemVerilog:

```
    output wire out
);
assign out = a + b;
assign valid_out = valid;
```

CppHDL:

```
_PORT(u<32>) out = _ASSIGN(a_in() + b_in());

void _assign()
{
    valid_out = _ASSIGN_REG(valid_reg);
}
```

Use `_ASSIGN(expr)` for expressions. Use `_ASSIGN_REG(reg_or_signal)` for direct storage bindings such as registers, logic values, memories, or ports whose final object reference is enough. Use `_ASSIGN_COMB(comb_func())` when assigning the result of a CppHDL combinational function. Even though `_ASSIGN_COMB()` captures the returned object by reference, the `comb_func()` call itself is still executed on demand when the port value is read. For loop-indexed assignments use `_ASSIGN_I`, `_ASSIGN_REG_I`, `_ASSIGN_COMB_I`, or the indexed forms such as `_ASSIGN_INDEXED((i,j,k), expr)` and `_ASSIGN_REG_INDEXED((i,j,k), object[i][j][k])`.

In the default single-clock flow, all SystemVerilog `always_ff` blocks for one module map into one CppHDL `_work(bool reset)` method. `_work()` computes next register values. It may contain the logic that would be split across several `always_ff` blocks in SystemVerilog. Multi-clock designs use one named work method per clock and edge, as described in the Clock Domain Crossing chapter.

SystemVerilog:

```
always_ff @(posedge clk) begin
    if (reset) count <= '0;
    else if (enable) count <= count + 1;
```

```
end

always_ff @(posedge clk) begin
    if (reset) valid <= 1'b0;
    else valid <= enable_in;
end
```

CppHDL:

```
reg<u<8>> count_reg;
reg<u1> valid_reg;

void _work(bool reset)
{
    if (reset) {
        count_reg.clr();
        valid_reg.clr();
        return;
    }

    if (enable_in()) {
        count_reg._next = count_reg + u<8>(1);
    }
    valid_reg._next = enable_in();
}
```

SystemVerilog `always_comb` blocks map to CppHDL combinational functions, usually named `*_comb_func()`. They calculate temporary combinational values from current inputs and current register values. The usual style is to store the result in a member variable and return it by reference, as shown in the root examples.

SystemVerilog:

```
always_comb begin
    hit = valid && tag == req_tag;
    read_data = hit ? line[word] : '0;
end
```

CppHDL:

```
bool hit_comb_func()
{
    return valid_reg && tag_reg == req_tag_in();
}

u<32> read_data_comb_func()
{
    return hit_comb_func() ? line_reg[word_in()] : u<32>(0);
}
```

CppHDL commits registers and memories in the mandatory `_strobe()` method. `_strobe()` is executed recursively for each module at the end of each clock evaluation. Register `.strobe()` calls and memory `.apply()` calls are only allowed in a strobe method, not in `_assign()`, a work method, or comb functions. Multi-clock designs use a separate named strobe method for each clock and edge.

```
void _strobe()
{
    count_reg.strobe();
    valid_reg.strobe();
    member._strobe();
}
```

Introduction

CppHDL is a C++ hardware definition language extension for digital integrated circuit development, designed for two purposes:

1. Building a full cycle of digital RTL development and testing using the C++ language
2. Allowing extremely fast simulation of RTL defined with blocking assignments

In all operations CppHDL works as a reflection of the SystemVerilog model, which means that, at every stage, a 100% register-to-register copy of any CppHDL code exists in the SystemVerilog domain. This live CppHDL to SystemVerilog conversion makes it possible to

- Connect CppHDL teams to classical verification and testing teams
- Deliver SV RTL to fabrication processes and tools or third-party companies

The main benefits of using C++ for RTL development are replacing slow **simulation** with compilation and execution that can be up to 100 times faster, while using a modern language that is accessible to more developers. The following properties of the C++ language provide a strong foundation for the RTL development process:

- Ability to use many professional IDEs and tools for development and debugging, including support for large project management
- C++ is one of the most popular and powerful programming languages in the world, with an extremely wide community
- C++ is precisely understood language by modern AI models and extremely useful in both rapid hardware verification and developmet
- CppHDL makes many of C++ developers accessible for chipmaking industry
- C++ is extremely fast in compilation and execution
- It is free and does not require paying for instances

RTL modeling using CppHDL includes verification and testing, providing the power and speed of the C++ language for modeling digital signaling and digital system interaction. It makes the verification process simpler, more flexible, faster, and purely programmatic.

CppHDL generates pure SystemVerilog output that is mostly a reflection of the C++ source, providing line-to-line visibility. It is even possible to apply patches synchronously to both RTL representations during finalization. Generated SystemVerilog files can be frozen at any moment and used as the main source code for the next stage of the ASIC/FPGA production process.

Who is CppHDL for

- IC development AI Agent's shepherds (agent does 100 times more iterations per day and understands C++ language better)
- Digital IC development teams (ASIC, IP, libraries, FPGA) - for faster development and testing of complex digital designs, free of charge
- Digital IC developers - to use modern C++ environments and smart IDEs, powerful C++ debug tools, static analysis, and linting
- Software developers who want to deliver hardware and use a powerful modern language with OOP, templates, abstraction, recursion, etc.
- CAD/tool developers (especially AI-coding/training) - 100 times more work cycles per day, with C++ being better learned by GPTs
- Talent seekers - involving speakers of a popular programming language in a variety of modern projects

What is the difference from other C++ to Verilog products

- CppHDL is not HLS; it is a representation of SystemVerilog, register to register and clock to clock, completely repeating model behavior line by line
- A CppHDL module is a simple single-process activity without waits for synchronization, notifications, `yield()`, streams, or cooperative multitasking

Limitations

- CppHDL supports only digital design components written using blocking assignments
- Multi-clock RTL and common digital CDC structures are supported, but analog behavior, physical implementation, and timing constraints require external CDC and implementation tools
- Timing- or power-critical sections should be isolated at the architectural level

CppHDL is intended to bring ease and speed to the development of various digital circuits such as controllers, multiplexers, cache and memory functions, mathematical functions, digital data processing, transmitting circuits, etc.

Requirements

CppHDL is delivered in two parts:

- C++ headers that contain definitions of CppHDL datatypes
- A conversion/linter tool that provides CppHDL to SystemVerilog conversion

Since CppHDL works as a reflection of the SystemVerilog RTL model, a strong understanding of SystemVerilog modeling techniques is required, in particular:

- Synchronous logic digital circuits
- Blocking and non-blocking assignments
- Combinational logic digital circuits
- Structures, packages, packed arrays, and unpacked arrays

CppHDL syntax

A CppHDL source file can be written as an `.h` header or a `.cpp` object file. Each header file should describe a module or auxiliary information such as datatypes, constants, interfaces, inline function definitions, etc.

CppHDL provides the ability to use C++ structures and custom datatypes in order to build a comfortable and consistent ecosystem for project development. All types and constants are translated into SystemVerilog datatypes during the conversion process.

The following example shows appropriate usage of C++ defines and structures in CppHDL.

```
#pragma once

#include "cpphdl.h"
#include "PrjConfig.h"

using namespace cpphdl;

#define    CMD_IDLE    0
#define    CMD_RESET   1
#define    CMD_CONFIG  2

struct CmdConfig
{
    unsigned cmd_id;
    unsigned units:6;
    unsigned flags:2;
    unsigned address;
}__PACKED;
static_assert (sizeof(CmdConfig) == 4, "struct CmdConfig size is not correct");
```

Module description format

Module definition is a C++ class with cpphdl::Module base, which includes:

1. Optional private zone with nested modules
2. Public zone with I/O ports definitions and *_assign*() function body
3. Optional private zone for registers and variables
4. Public zone with *_work(reset)*, *_strobe*(), *_work_neg(reset)*, *_strobe_neg*() and combinational functions bodies

In the following block of code a simple FIFO model RTL shown as a basic CppHDL example:

```

#pragma once

#include "cpphdl.h"
#include "Memory.cpp"
#include <print>

using namespace cpphdl;

template<size_t FIFO_WIDTH_BYTES, size_t FIFO_DEPTH, bool SHOWAHEAD = true>
class Fifo : public Module
{
    Memory<FIFO_WIDTH_BYTES,FIFO_DEPTH,SHOWAHEAD> mem;

public:
    _PORT(bool)                         write_in;
    _PORT(logic<FIFO_WIDTH_BYTES*8>)    write_data_in;

    _PORT(bool)                         read_in;
    _PORT(logic<FIFO_WIDTH_BYTES*8>)    read_data_out  = mem.read_data_out;

    _PORT(bool)                         empty_out      = _ASSIGN_COMB( empty_comb_func()
    ↪  );
    _PORT(bool)                         full_out       = _ASSIGN_COMB( full_comb_func()
    ↪  );
    _PORT(bool)                         clear_in       = _ASSIGN( false );
    _PORT(bool)                         afull_out      = _ASSIGN_REG( afull_reg );

    bool                        debugen_in;

private:
    reg<u<clog2(FIFO_DEPTH)>> wp_reg;
    reg<u<clog2(FIFO_DEPTH)>> rp_reg;
    reg<u1> full_reg;
    reg<u1> afull_reg;

public:

    void _assign()
    {
        mem.write_data_in = write_data_in;
        mem.write_in      = write_in;
        mem.write_mask_in = _ASSIGN( 0xFFFFFFFFFFFFFFFFULL );
        mem.write_addr_in = _ASSIGN_REG( wp_reg );
        mem.read_in       = read_in;
        mem.read_addr_in  = _ASSIGN_REG( rp_reg );
        mem.__inst_name = __inst_name + "/mem";
        mem.debugen_in  = debugen_in;
        mem._assign();
    }

    bool full_comb;
    bool& full_comb_func()
```

```
52      {
53          return full_comb = (wp_reg == rp_reg) && full_reg;
54      }
55
56      bool empty_comb;
57      bool& empty_comb_func()
58      {
59          return empty_comb = (wp_reg == rp_reg) && !full_reg;
60      }
61
62      void _work(bool reset)
63      {
64          mem._work(reset);
65
66          if (debugen_in) {
67              std::print("{:s}: input: ({})({}, output: ({})({}, wp_reg: {}, rp_reg: {},
                ↪  full: {}, empty: {}, reset: {}\n", __inst_name,
68                  (int)write_in(), write_data_in(), (int)read_in(), read_data_out(),
                        ↪  wp_reg, rp_reg, (int)full_out(), (int)empty_out(), reset);
69          }
70
71          if (reset) {
72              wp_reg.clr();
73              rp_reg.clr();
74              full_reg.clr();
75              afull_reg.clr();
76              return;
77          }
78
79          if (write_in()) {
80
81              if (full_comb_func() && !read_in()) {
82                  std::print("{:s}: writing to a full fifo\n", __inst_name);
83                  exit(1);
84              }
85              if (!full_comb_func() || read_in()) {
86                  wp_reg._next = wp_reg + 1;
87              }
88              if (wp_reg._next == rp_reg) {
89                  full_reg._next = 1;
90              }
91          }
92
93          if (read_in()) {
94
95              if (empty_comb_func()) {
96                  std::print("{:s}: reading from an empty fifo\n", __inst_name);
97                  exit(1);
98              }
99              if (!empty_comb_func()) {
100                 rp_reg._next = rp_reg + 1;
101             }
102             if (!write_in()) {
103                 full_reg._next = 0;
```

```
            }
        }

        if (clear_in()) {
            wp_reg._next = 0;
            rp_reg._next = 0;
            full_reg._next = 0;
        }

        afull_reg._next = full_reg || (wp_reg >= rp_reg ? wp_reg - rp_reg : FIFO_DEPTH -
        ↪  rp_reg + wp_reg) >= FIFO_DEPTH/2;
    }

    void _strobe()
    {
        mem._strobe();
        wp_reg.strobe();
        rp_reg.strobe();
        full_reg.strobe();
        afull_reg.strobe();
    }
};
```

- A module class definition can use template parameters
- Built-in C++ types such as `bool`, `unsigned`, `unsigned long`, etc. are allowed in all places except `reg<>`
- Each of the *_assign*(), *_work*(), and *_strobe*() functions should call the corresponding functions of nested modules
- Only the *reg_name._next* value can be changed outside reset. Both `reg` and *reg._next* values can be used on the right side of expressions
- During reset, *.clr*() and *.set(val)* methods are used to set both current and next register values
- CppHDL replaces *[f]printf*(), *std::print*, *$write*(), and *exit*() functions with their SV equivalents, and parameters are converted appropriately

Input/output ports

All ports are `cpphdl::function_ref<data_type>` objects, declared through `_PORT(data_type)`. A port stores either a value-producing expression or a reference-producing binding and caches the resolved value for the current `_system_clock`. This allows an entire combinational function chain to be recalculated on demand without exposing heap-owning `std::function` as the port API.

- Macro *_PORT(`data_type`)* allows simple port declaration.

- Macro *_ASSIGN(`any_cpp_expression`)* represents a Verilog assign expression. The return type can be cast, and the compiler checks the lambda return type.
- Macro *_ASSIGN_REG(`member_or_signal`)* is a fast replacement for *_ASSIGN*() when the value is a persistent variable in the class.
- Macro *_ASSIGN_COMB(`comb_func()`)* is used for combinational functions that return a reference to a persistent result variable.

The following naming convention is used for input/output ports (use `_in` ending or `_out` ending):

- `port_in` or `obj_port_in` - generic input port name
- `port_out` or `obj_port_out` - generic output port name
- or longer name, ending with *_in* or *_out*

It is recommended to initialize all output ports directly in the class description when possible. In more complex situations, it is recommended to initialize output ports in a special *_assign*() function. Input ports can be assigned an explicit *_ASSIGN(0)* tie-off when the surrounding native simulation does not provide a source.

NOTE! To build a complex bus interface between a CppHDL module and a third-party SV module, use packed *structs* to achieve proper `<8`bit fields packing.

Clock and reset

The default flow uses one clock named *clk*. Multi-clock designs declare a primary clock and one or more secondary clocks on the `cpphdl` command line. Each declared clock becomes a module port and has its own work and strobe methods. Reset is the main *reset* parameter of each work function. Named multi-clock processes support synchronous reset and active-high asynchronous reset assertion. Asynchronous-reset method naming, ownership, and release requirements are described in the Clock Domain Crossing chapter.

Variables list

Variables are module class members and can be of one of 3 types:

- **registers**
- **combinational**
- **temporary**

Registers are of type **reg<TYPE>** and contain value, updated on strobing clock edge. To access next value of a register the *reg_name._next* property is used. It is recommended to give register names with a *reg* suffix in case when register is used as output port or in parent modules.

- Registers can carry structs, arrays, and single values.
- Combinational variable can be of any type.
- Variables can be declared inside methods/functions, but declarations are allowed only at the very beginning of the method/function, as in C.

Connect method

The *_assign*() method is used to assign nested instance inputs to data sources in the module and back again.

Work method

The work method can make changes to registers and temporary variables. Only *._next* value of registers should be changed directly.

- Work method can call other methods to make code well-structured.
- Methods with return values become SystemVerilog functions; methods with `void` return become Verilog tasks.
- It is possible to change registers only from void methods.
- Methods can take references to registers as parameters.

Strobe method

The strobe function should contain all registers of the module and call `.strobe()` functions for them. Also, `_strobe()` should be called for each nested instance of the class. Forgotten registers will be reported by *cpphdl* tool. In a multi-clock design, each register or memory must be committed by exactly one clock-and-edge-specific strobe method.

Comb methods

Combinational methods represent Verilog combinational logic functions. All combinational methods should comply with the following requirements:

- The name of the function should contain *_comb_func*() suffix
- A corresponding variable should be defined in the module class: *var_name_comb*
- The combinational function should calculate and assign a value to the *var_name_comb* variable, then return a reference to it
- **NOTE!** The global variable `_system_clock` is used to invalidate cached port and lazy-combinational values after a simulation step. A native testbench must define it and increment it as shown in the examples.

It will be converted to a corresponding SystemVerilog variable and `always_comb` block during conversion.

It is important to avoid loops in combinational function call chains.

Data types

To repeat SystemVerilog behavior, CppHDL implements a number of basic data types, each corresponding to a specific SystemVerilog datatype. Currently, the list of CppHDL datatypes includes:

- *logic<WIDTH>* - any width variable, optimized for bit-access
- *u<WIDTH>*, u1, u8, u16, u32, u64 - unsigned variables
- *i8*, *i16*, *i32*, *i64* - fixed-width signed native aliases
- *reg<TYPE>* - register definition, works only with CppHDL types or any structs
- *array<COUNT,TYPE,PACKED=false>* - fixed-size packed or unpacked array
- *array2D*, *array3D*, *array4D* - multidimensional array aliases
- *memory<TYPE,ROW_SIZE,DEPTH>* - deferred-write memory container committed by `apply()`
- *Cat{...}* - concatenation helper that maps to SystemVerilog `{...}` expressions

RTL index widths

Indexes and loop variables that are converted into RTL must not be 64-bit. Some synthesis and RTL-processing tools cannot reliably elaborate 64-bit array selectors and may report width errors, fail optimization, or terminate with an internal error. Use `uint32_t`, a narrower native unsigned type, or an explicitly sized CppHDL type such as `u<WIDTH>` according to the required index range. In particular, do not use `size_t` for synthesizable array indexes or loop variables on 64-bit hosts. `size_t` remains appropriate for compile-time template parameters and other C++ elaboration-only values that do not become RTL signals.

logic<WIDTH>

logic<> is the basic type of the CppHDL toolchain, representing the SystemVerilog `logic` type. It is universal, can be of any width, and can be used as a standalone variable or inside a reg<> construction.

Example usage as a variable:

```
1 logic<MEM_WIDTH_BYTES*8> data_out_comb;
```

Example usage as a port:

```
_PORT(logic<MEM_WIDTH_BYTES*8>) data_in;
```

The logic<> type provides read/write access to individual bits using *operator[]* and to partial bitmaps using the *.bits(hi,lo)* method:

```
buffer1_byteenable._next[addr_sub+i] = 1;
host_addr.bits(39,32) = s_writedata_in() >> 32;
```

The *.bits(hi,lo)* arguments can be expressions, so indexed slices such as `bits(word * 16 + 15, word * 16)` are supported and converted to SystemVerilog indexed part-selects. The slice width must be statically implied by the expression shape: use the same base expression on both sides plus a constant width, keep `hi >= lo`, and keep the selected range inside the `logic<>` width. Dynamic indexed slices are intended for contiguous read/write fields, not for arbitrary variable-width ranges.

Examples from `tests/datatypes/LogicBitsIndexing.cpp`:

```
logic<128> source_comb;
logic<16> direct_comb;
logic<8> byte_comb;

direct_comb = source_comb.bits(word * 16 + 15, word * 16);
byte_comb = source_comb.bits(word * 8 + 7, word * 8);
```

Writing indexed slices is supported as well:

```
logic<128> edited_comb;

edited_comb.bits(word * 16 + 15, word * 16) =
    logic<16>((uint64_t)seed_in() ^ 0x55aa);

edited_comb.bits((word + 1) * 16 + 15, (word + 1) * 16) =
    logic<16>((uint64_t)seed_in() ^ 0xaa55);

edited_comb.bits(word * 8 + 7, word * 8) =
    logic<8>((uint64_t)seed_in() ^ 0x5a);
```

u<WIDTH>

u<> is a basic unsigned value with a width from 1 through 64 bits. It supports the usual arithmetic and bitwise operators and can be cast to a logic<> variable. Example usage of *u<>*:

```
u<STEPS_SIZE> cmd_steps;
```

Cat{...}

Cat{...} builds a packed concatenation from CppHDL values and maps directly to a SystemVerilog concatenation expression. Arguments are placed from high bits to low bits in the same order as SystemVerilog `{a, b, c}`. The result width is inferred from the argument types. Supported operands include `logic<WIDTH>`, `u<WIDTH>`, fixed unsigned aliases such as `u8` and `u32`, and `reg<T>` values whose stored type is supported.

Example assignment to a port:

```
u<4> byteenable;
u<5> address;
u<8> data;

void _assign()
{
    buffer.valid_in = _ASSIGN(Cat{byteenable, address, data});
}
```

This is emitted as a SystemVerilog concatenation:

```
assign buffer__valid_in = {byteenable, address, data};
```

The same helper can be used for register next-state assignment:

```
reg<logic<17>> packet_reg;
reg<u<4>> reg_byteenable;
reg<u<5>> reg_address;
reg<u<8>> reg_data;

void _work(bool reset)
{
    packet_reg._next = Cat{reg_byteenable, reg_address, reg_data};
}
```

The destination type must be wide enough for the concatenation result. Width mismatches are checked by the C++ type system and by the generated SystemVerilog tools.

u1, u8, u16, u32, u64

u1, *u8*, *u16*, *u32*, and *u64* are fixed-width convenience classes that map to 1-, 8-, 16-, 32-, and 64-bit unsigned RTL values, respectively.

reg<TYPE>

The reg<> template is intended to make a variable a register. It adds the *._next* property, which is changed in a *_work*() function, as well as the ._strobe() method, which synchronizes the current value with the next value. It should not be used as a port definition, but it can provide data to a port. Examples of reg<> usage are provided below:

```
reg<State> state;
reg<u16> size;
reg<array<WIDTH/8, u8>> buffer1;
reg<logic<WIDTH/8>> buffer1_byteenable;
```

```
state_struct._next.steps = state_struct.steps - 1;
if (state_struct.steps == 0) {
    state_struct._next.steps = 255;
}

size._next = 0xFFFF;

buffer1._next |= buffer1_precalc;

buffer1_byteenable._next[i] = buffer2_byteenable[i];

mask._next.bits((i+1)*32-1,i*32) = 0;
```

array<COUNT, TYPE, PACKED>

The `array<>` type stores `COUNT` elements of `TYPE` and can be used with `reg<>`. `PACKED` defaults to `false`; pass `true` when a packed CppHDL representation is required. The `array2D`, `array3D`, and `array4D` aliases take all dimensions first, followed by the element type and optional packed flag.

```
reg<array<WIDTH/8, u8>> buffer1;
_PORT(array<WIDTH/8, u8>) avmm_writedata_out = _ASSIGN_REG(buffer1);

array2D<4, 8, u16> matrix;
array3D<2, 3, 4, u8, true> packed_volume;
```

memory<TYPE, ROW_SIZE, DEPTH>

The *memory*<> type is developed for optimal access performance to registered memory, with the ability to change one word per clock cycle. It cannot be used as a port. It uses the *apply*() method for strobing data. The following example shows how memory<> should be used to organize simple memory with one read and one write port.

```cpp
#pragma once

#include "cpphdl.h"
#include <print>

using namespace cpphdl;

template<size_t MEM_WIDTH_BYTES, size_t MEM_DEPTH, bool SHOWAHEAD = true>
class Memory : public Module
{
    reg<logic<MEM_WIDTH_BYTES*8>> data_out_reg;
    memory<u8,MEM_WIDTH_BYTES,MEM_DEPTH> buffer;

public:
    _PORT(u<clog2(MEM_DEPTH)>)       write_addr_in;
    _PORT(bool)                      write_in;
    _PORT(logic<MEM_WIDTH_BYTES*8>)  write_data_in;
    _PORT(logic<MEM_WIDTH_BYTES>)    write_mask_in;

    _PORT(u<clog2(MEM_DEPTH)>)       read_addr_in;
    _PORT(bool)                      read_in;
    _PORT(logic<MEM_WIDTH_BYTES*8>)  read_data_out = _ASSIGN_COMB( data_out_comb_func() );

    bool                        debugen_in;

    void _assign() {}

    logic<MEM_WIDTH_BYTES*8> data_out_comb;
    logic<MEM_WIDTH_BYTES*8>& data_out_comb_func()
    {
        if (SHOWAHEAD) {
            data_out_comb = buffer[read_addr_in()];
        }
        else {
            data_out_comb = data_out_reg;
        }
        return data_out_comb;
    }

    void _work(bool reset)
    {
        uint32_t i;
        logic<MEM_WIDTH_BYTES*8> mask;

        if (write_in()) {
            mask = 0;
            for (i=0; i < MEM_WIDTH_BYTES; ++i) {
                mask.bits((i+1)*8-1,i*8) = write_mask_in()[i] ? 0xFF : 0 ;
            }
            buffer[write_addr_in()] = (buffer[write_addr_in()]&~mask) | (write_data_in()&mask);
        }
```

```
52
53          if (!SHOWAHEAD) {
54              data_out_reg._next = buffer[read_addr_in()];
55          }
56
57          if (debugen_in) {
58              std::print("{:s}: input: ({})[{}@{}({}), output: ({})[{}@{}\n", __inst_name,
59                  (int)write_in(), write_data_in(), write_addr_in(), write_mask_in(),
60                  (int)read_in(), read_data_out(), read_addr_in());
61          }
62      }
63
64      void _strobe()
65      {
66          buffer.apply();
67          data_out_reg.strobe();
68      }
69  };
70
```

Interfaces

Interfaces are special structures which consist of ports of any direction. There is no need for separate "driver" and "responder" interface types: both sides use the same `Interface` type, and the port suffix (`_in` or `_out`) describes signal direction relative to each module. For example, `source_out.valid_in` and `sink_in.valid_in` are both the valid signal driven by the source and consumed by the sink. During SystemVerilog conversion these names are flattened, for example `source_out.valid_in` becomes `source_out__valid_out` on the driver module.

```
1
2   template<size_t DATAWIDTH>
3   struct DataValidReady
4   {
5       bool valid;
6       bool ready;
7       logic<DATAWIDTH> data;
8   };
9
10  template<size_t DATAWIDTH>
11  struct DataValidReadyIf : public Interface
12  {
13      _PORT(bool) valid_in;
14      _PORT(bool) ready_out;
15      _PORT(logic<DATAWIDTH>) data_in;
16  };
17
18  template<size_t DATAWIDTH>
19  class VRDriver : public Module
```

```
{
public:
    DataValidReadyIf<DATAWIDTH> source_out;

private:
    reg<u1> valid_reg;
    reg<logic<DATAWIDTH>> data_reg;

public:
    void _assign()
    {
        source_out.valid_in = _ASSIGN_REG(valid_reg);
        source_out.data_in = _ASSIGN_REG(data_reg);
    }

    void _work(bool reset)
    {
        if (reset) {
            valid_reg.clr();
            data_reg.clr();
            return;
        }

        valid_reg._next = 1;
        if (!valid_reg || source_out.ready_out()) {
            data_reg._next = data_reg + logic<DATAWIDTH>(1);
        }
    }

    void _strobe()
    {
        valid_reg.strobe();
        data_reg.strobe();
    }
};

template<size_t DATAWIDTH>
class VRResponder : public Module
{
public:
    DataValidReadyIf<DATAWIDTH> sink_in;

private:
    reg<u1> ready_reg;
    reg<logic<DATAWIDTH>> last_data_reg;

public:
    void _assign()
    {
        sink_in.ready_out = _ASSIGN_REG(ready_reg);
    }

    void _work(bool reset)
    {
```

```
        if (reset) {
            ready_reg.clr();
            last_data_reg.clr();
            return;
        }

        ready_reg._next = 1;
        if (sink_in.valid_in() && sink_in.ready_out()) {
            last_data_reg._next = sink_in.data_in();
        }
    }

    void _strobe()
    {
        ready_reg.strobe();
        last_data_reg.strobe();
    }
};

class TestValidReady : public Module
{
    VRDriver<32> driver;
    VRResponder<32> responder;

public:
    void _assign()
    {
        driver.__inst_name = __inst_name + "/driver";
        responder.__inst_name = __inst_name + "/responder";
        assignIf(driver, responder, driver.source_out, responder.sink_in);
    }
};

```

`assignIf(modA, modB, ifA, ifB)` connects two interface instances and calls the modules' `_assign()` methods in the order needed for bidirectional interface signals. This is important for interfaces such as valid-ready, where one module drives `valid` and `data`, while the other drives `ready`. Use it from a parent module or test wrapper after setting instance names. Check `examples/axi/Axi4MuxFromSlave.cpp`, `examples/axi/Axi4MuxToMaster.cpp`, and `tests/interface/ValidReady.cpp` for larger examples.

Clock Domain Crossing (CDC)

CppHDL can generate and simulate modules with multiple independent clock domains. The converter creates one clock input port and one sequential SystemVerilog process for each declared clock and edge. CppHDL does not make an unsafe crossing safe automatically: the RTL author must select an appropriate synchronizer, handshake, or asynchronous FIFO for every value crossing between domains.

Declaring clock domains

Declare clocks before the source file in the `cpphdl` command line:

```
cpphdl \
    --primary_clock fast_clk 100000000 \
    --secondary_clock slow_clk 40000000 \
    TwoClocksCdc.cpp -I../include
```

The clock options follow these rules:

- `--primary_clock <clk_name> <frequency>` declares the design's primary clock.
- `--secondary_clock <clk_name> <frequency>` may be repeated for additional clocks.
- A secondary clock requires a primary clock.
- Clock names must be valid, unique, non-keyword C++ identifiers.
- Frequencies are positive integers. Use one consistent unit, normally hertz.
- The primary clock frequency must be greater than or equal to every secondary clock frequency.
- With no clock options, CppHDL retains the legacy `clk`, `_work(bool reset)`, and `_strobe()` flow.
- A single named primary clock changes the generated clock port name but retains the legacy `_work(bool reset)` and `_strobe()` methods.
- With two or more clocks, clock declarations are design-global. Every generated module receives every declared clock port.

The frequency values describe and validate the clock topology. They do not create a clock source and do not automatically schedule native C++ simulation edges. A testbench must drive each clock with the required period, phase, and edge ordering.

Clock process methods

For every declared clock `<clk_name>` in a multi-clock design, every generated module must define this positive-edge pair:

```
void _work_<clk_name>(bool reset);
void _strobe_<clk_name>();
```

An optional negative-edge process requires both methods:

```
void _work_neg_<clk_name>(bool reset);
void _strobe_neg_<clk_name>();
```

Defining only one method of a negative-edge pair is an error. A domain with no positive-edge state still requires an empty positive-edge work/strobe pair because clocks are currently design-global.

The following example transfers a single-bit level through a two-register destination-domain synchronizer:

```
class LevelCdc : public Module
{
public:
    _PORT(bool) source_in;
    _PORT(bool) synchronized_out = _ASSIGN_REG(sync2_reg);

private:
    reg<u1> source_reg;
    // (* ASYNC_REG = "TRUE" *)
    reg<u1> sync1_reg;
    // (* ASYNC_REG = "TRUE" *)
    reg<u1> sync2_reg;

public:
    void _work_fast_clk(bool reset)
    {
        source_reg._next = source_in();
        if (reset) {
            source_reg.clr();
        }
    }

    void _strobe_fast_clk()
    {
        source_reg.strobe();
    }

    void _work_slow_clk(bool reset)
    {
        sync1_reg._next = source_reg;
        sync2_reg._next = sync1_reg;
        if (reset) {
            sync1_reg.clr();
```

```
            sync2_reg.clr();
        }
    }

    void _strobe_slow_clk()
    {
        sync1_reg.strobe();
        sync2_reg.strobe();
    }

    void _assign() {}
};
```

The converter emits separate clock ports and sequential blocks equivalent to:

```
module LevelCdc (
    input wire fast_clk,
    input wire slow_clk,
    input wire reset
    // ports omitted
);

always_ff @(posedge fast_clk) begin
    _work_fast_clk(reset);
end

always_ff @(posedge slow_clk) begin
    _work_slow_clk(reset);
end
endmodule
```

Register and memory ownership

A register or memory belongs to exactly one clock and edge. Its next value must be written by the corresponding work method, and it must be committed by the matching strobe method. The converter rejects these structural errors:

- A register is written by a work method but omitted from its matching strobe method.
- A register is strobed by more than one clock or edge.
- A positive-edge work/strobe method required by a declared clock is missing.
- Only one method of an optional negative-edge pair is present.

Do not write the same storage from two domains. Cross a control value into the destination domain first, then let destination-owned logic update destination-owned storage. For memory-backed asynchronous FIFOs, one domain owns the write operation while the other reads through the FIFO's defined read path; Gray-coded pointers cross between the domains.

Reset ownership and sequencing

A generated multi-clock module has one shared `reset` input, driven by the parent module or testbench and connected to every nested module. Reset is a level, not a one-time event and not a transaction consumed by one clock. Each clock process samples that same level only on its own active edge and resets only the storage owned by that clock and edge. Consequently, a fast-clock edge does not reset slow-domain storage, and there is no second reset of fast-domain storage when the slow clock later samples reset.

Reset is complete only after `reset` has remained asserted across at least one active edge of every clock-and-edge process that owns storage. If both positive- and negative-edge state exist, both edges must observe reset. Holding reset for additional edges repeats the reset assignments and must be harmless. A one-primary-clock-cycle reset pulse is unsafe because a slower or stopped clock can miss it entirely.

Use this sequence in native CppHDL and Verilator testbenches:

1. Quiesce normal transaction inputs and keep all declared clocks running.
2. Assert the shared `reset` input before any reset edge is evaluated.
3. Drive enough complete cycles that every positive- and negative-edge owner observes asserted reset. Designs with initialization state machines may require more cycles.
4. Keep reset asserted until the slowest domain has observed it.
5. Deassert reset according to each domain's synchronous timing requirements. Use a per-domain synchronization chain when reset release can arrive asynchronously.
6. Resume traffic only after every required domain-local reset-release indication is active.

CppHDL statically lints the structural part of this protocol. Every named work method must have the `void _work_<clk_name>(bool reset)` signature, where the C++ parameter may be unnamed, every strobe method must have a `void _strobe_<clk_name>()` signature, required method pairs must exist, and each register must have one clock/edge owner. The converter cannot prove that a testbench holds reset long enough, that every register is intentionally reset, that clocks continue running during reset, or that an external reset meets recovery/removal timing. Those properties require dynamic regression checks, generated-SystemVerilog assertions, and implementation CDC/reset-domain-crossing analysis.

`test_shared_reset_sampling_per_domain()` in `tests/cdc/TwoClocksCdc.cpp` verifies that advancing only the fast clock resets only fast-owned state, that repeated asserted edges are idempotent, and that reset becomes complete after the slow domain also receives an active edge. `test_synchronized_reset_release()` separately verifies two-stage domain-local release in both native CppHDL and Verilator flows.

Native and Verilator simulation

A native CppHDL testbench schedules every clock independently. On an active edge, call the work method before the matching strobe method:

```
if (fast_positive_edge) {
    dut._work_fast_clk(reset);
    dut._strobe_fast_clk();
}
if (slow_positive_edge) {
    dut._work_slow_clk(reset);
    dut._strobe_slow_clk();
}
if (fast_negative_edge) {
    dut._work_neg_fast_clk(reset);
    dut._strobe_neg_fast_clk();
}
```

A Verilator testbench drives the generated clock ports and calls `eval()` after every level change. Tests should use different periods and a nontrivial phase relationship so simultaneous edges are not assumed. Reset behavior must be exercised independently in every domain. Compare externally visible transactions rather than internal scheduling details when checking native and Verilator equivalence.

CDC implementation patterns

Use a CDC structure that matches the transferred information:

- Stable single-bit levels: use at least a two-register destination-domain synchronizer.
- Multi-bit counters and FIFO pointers: convert to Gray code, synchronize the Gray bus, then consume it in the destination domain.
- Narrow pulses: encode the event as a toggling bit, synchronize it, and detect a toggle in the destination domain.
- Coherent multi-bit payloads: hold data stable while a request/acknowledge handshake crosses the domains.
- Streams: use a dual-clock asynchronous FIFO with domain-local binary pointers and synchronized Gray pointers.
- Reset release: assert reset through clocked logic and release it through a per-domain synchronizer.

Synchronizer registers can carry synthesis attributes through an adjacent CppHDL annotation comment:

```
// (* ASYNC_REG = "TRUE" *)
reg<u1> request_sync1_reg;
// (* ASYNC_REG = "TRUE" *)
reg<u1> request_sync2_reg;
```

This emits the corresponding `ASYNC_REG` attributes in generated SystemVerilog. The attribute assists synthesis and implementation tools, but does not replace CDC timing constraints or CDC signoff.

Covered CDC Features

Each item has dedicated regression coverage in `tests/cdc/TwoClocksCdc.cpp` or `tests/reset/AsyncReset.cpp`:

- Independent clocks with unrelated phase/frequency
- Generated clock ports and separate edge blocks
- Exclusive register ownership per clock/edge
- Two-flop single-bit synchronization
- Gray-coded multi-bit counter transfer
- Toggle-based narrow-pulse transfer
- Request/acknowledge coherent data mailbox
- Memory-backed asynchronous FIFO
- Shared-reset sampling and completion across all clock domains
- Active-high asynchronous reset assertion for positive- and negative-edge processes
- Synchronized reset release
- Positive/negative-edge processes
- `ASYNC_REG` synthesis attributes

The expanded regression passes in both native CppHDL and Verilator flows. Existing CLI failure tests and the legacy `code_VarInit` test also pass. No additional converter defect was exposed by this coverage.

Current Limits

The following behavior cannot presently be validated or fully represented by CppHDL:

- Analog metastability behavior and mean time between failures (MTBF)
- Physical synchronizer placement and routing skew
- SDC false-path, multicycle, and generated-clock constraints
- Automatic detection of unsafe raw buses, reconvergence, or lost pulses
- Active-low reset polarity and independent per-domain reset ports; asynchronous handlers currently use the shared active-high `reset`
- Dynamic clock gating or clock multiplexing
- Jitter, drift, and precise phase relationships
- Module-local clock subsets or clock-name remapping; clocks are currently design-global
- Power-domain isolation and level-shifter behavior

These limits require dedicated CDC analysis, static timing analysis, physical implementation constraints, and hardware signoff outside CppHDL.

Asynchronous Reset

Named clock processes support asynchronous assertion through optional active-high reset handlers:

```
void _reset_pos_fast_clk()
{
    fast_state_reg.clr();
}

void _reset_neg_fast_clk()
{
    fast_neg_state_reg.clr();
}
```

`_reset_pos_<clk_name>()` belongs to the positive clock-edge process, while `_reset_neg_<clk_name>()` belongs to the negative clock-edge process. The suffix describes the clock edge, not reset polarity. Generated RTL uses the shared active-high `reset`:

```
always_ff @(posedge fast_clk or posedge reset) begin
    if (reset)
        _reset_pos_fast_clk();
    else
        _work_fast_clk(reset);
end
```

Reset handlers must return `void`, take no arguments, and modify only registers owned and strobed by the matching clock edge. A negative-edge handler requires matching `_work_neg_<clk_name>(bool)` and `_strobe_neg_<clk_name>()` methods.

In native simulation, call the reset handler followed by its matching strobe method when reset is asserted. In Verilator, change `reset` from `0` to `1` and call `eval()`; no clock edge is required. Reset release must still satisfy each clock domain's recovery/removal requirements, normally through synchronized release logic.

See `tests/reset/AsyncReset.cpp` for native and Verilator examples.

VCD dumping

Native CppHDL simulation can write a Value Change Dump file with the lightweight `VcdFile` helper from `cpphdl_vcd.h`. VCD dumping is controlled by the C++ testbench; the converter does not automatically discover signals or sample simulation time.

Register each signal with a stable name, its RTL width in bits, and a pointer to storage that remains alive for the complete trace:

```
#include <cpphdl.h>
#if !defined(SYNTHESIS)
#include <cpphdl_vcd.h>
#endif

using namespace cpphdl;

long _system_clock = -1;

class Counter : public Module
{
    reg<u<8>> count_reg;

public:
    _PORT(u<8>) count_out = _ASSIGN_REG(count_reg);

    void _work(bool reset)
    {
        if (reset) {
            count_reg.clr();
        }
        else {
            count_reg._next = count_reg + 1;
        }
    }

    void _strobe()
    {
        count_reg.strobe();
    }

    void _assign() {}

#if !defined(SYNTHESIS)
    void add_vcd_signals(VcdFile& vcd, const std::string& prefix)
    {
        vcd.signals.push_back({prefix + "count_reg", 8, &count_reg});
    }
#endif
};
```

Create the file after all signals have been registered, then call `sample()` at monotonically increasing timestamps:

```
Counter dut;
VcdFile vcd;

dut._assign();
dut.add_vcd_signals(vcd, "dut.");
vcd.create("output.vcd");

```

```
dut._work(true);
vcd.sample(0);

for (unsigned cycle = 1; cycle <= 100; ++cycle) {
    dut._strobe();
    ++_system_clock;
    dut._work(false);
    vcd.sample(cycle);
}
```

`VcdFile::create()` currently emits a `1ns` timescale. The argument to `sample(time_ns)` is therefore the VCD timestamp in nanoseconds; it does not advance the model. For multiple clocks, sample after every clock transition or scheduled event and use timestamps that represent the testbench's actual edge schedule.

The signal pointer must refer to contiguous raw storage such as a CppHDL scalar, `logic<>`, `reg<>`, or array whose layout is suitable for raw bit reading. Do not register a temporary expression or a `_PORT`/`function_ref` object. If a port value is needed, copy it into persistent testbench storage before sampling. Limit long traces explicitly because VCD files grow quickly. `examples/basic/Buffer.cpp`, `examples/basic/Fifo.cpp`, and `examples/basic/Memory.cpp` contain complete native examples and cap the number of samples.

This helper traces the native CppHDL model. To trace internal signals of generated RTL under Verilator, build the generated SystemVerilog with Verilator's `--trace` option. The testbench must enable tracing before time advances, register the model, and call `dump()` after every evaluated clock transition:

```
#include <verilated.h>
#include <verilated_vcd_c.h>
#include "VTop.h"

int main(int argc, char** argv)
{
    Verilated::commandArgs(argc, argv);
    Verilated::traceEverOn(true);

    VTop dut;
    VerilatedVcdC trace;
    dut.trace(&trace, 99);
    trace.open("output.vcd");

    vluint64_t time = 0;
    dut.reset = 1;
    for (unsigned cycle = 0; cycle < 100; ++cycle) {
        dut.clk = 0;
        dut.eval();
        trace.dump(time++);

        dut.clk = 1;
```

```
        dut.eval();
        trace.dump(time++);

        if (cycle == 1) {
            dut.reset = 0;
        }
    }

    trace.close();
    dut.final();
}
```

For example, pass `--trace` alongside the usual Verilator generation options before building the generated model. Replace `VTop` and `VTop.h` with the selected top-module model. The repository's normal `VerilatorCompile()` test helper does not enable internal tracing automatically.

CppHDL SV Conversion tool

The main purposes of the *cpphdl* tool are to

- Provide conversion of CppHDL code to SystemVerilog models
- Check dependencies in combinational chains and forgotten strobe calls in CppHDL source code

Structures

- Each structure or union is converted into SystemVerilog package in a separate file
- The order of fields is reversed due to differences between C++ and SystemVerilog
- Anonymous structures and unions declared inside other structures get 'anon' name substitution
- CppHDL aligns non-bitfield structure members to byte boundaries and emits hidden `_alignN` fields when padding is needed
- Packed unions use the largest branch size; smaller struct/union branches get hidden `_padN` fields so all union alternatives have the same packed width
- `cpphdl::array<N,T>` fields are emitted as packed SystemVerilog arrays inside the generated struct package
- C++ enums are emitted with a four-state SystemVerilog `logic` base preserving the C++ underlying width and signedness

Example from `tests/structs/ArrayInStruct.cpp`:

```
struct ArrayPayload
{
    unsigned prefix:4;
    array<3, u8> bytes;
    unsigned mid:3;
    array<1, u16> halfs;
    unsigned tail:5;
} __PACKED;
```

Generated SystemVerilog:

```
package ArrayPayload_pkg;

typedef struct packed {
    logic[3-1:0] _align0;
    logic[5-1:0] tail;
    logic[1-1:0][16-1:0] halfs;
    logic[5-1:0] _align2;
    logic[3-1:0] mid;
    logic[3-1:0][8-1:0] bytes;
    logic[4-1:0] _align1;
    logic[4-1:0] prefix;
} ArrayPayload;

endpackage
```

Example from `tests/structs/StructAlignment.cpp`:

```
struct TinyBits
{
    unsigned a:1;
    unsigned b:2;
    unsigned c:3;
} __PACKED;

struct MixedBits
{
    unsigned flag:1;
    unsigned code:4;
    u<3> state;
    unsigned tail:2;
} __PACKED;

struct OuterBits
{
    unsigned head:3;
    TinyBits tiny;
    unsigned mid:5;
```

```
    MixedBits mixed;
    u<4> nibble;
    unsigned last:1;
} __PACKED;
```

Generated SystemVerilog:

```
package OuterBits_pkg;
import TinyBits_pkg::*;
import MixedBits_pkg::*;

typedef struct packed {
    logic[7-1:0] _align0;
    logic[1-1:0] last;
    logic[4-1:0] _align3;
    logic[4-1:0] nibble;
    MixedBits mixed;
    logic[3-1:0] _align2;
    logic[5-1:0] mid;
    TinyBits tiny;
    logic[5-1:0] _align1;
    logic[3-1:0] head;
} OuterBits;

endpackage
```

Packed union branches are also normalized to a common size:

```
struct UnionStruct
{
    unsigned sa:4;
    u<6> sb;
    unsigned sc:1;
} __PACKED;

union UnionWithStruct
{
    struct {
        unsigned us0:2;
        UnionStruct nested;
        unsigned us1:3;
    } __PACKED branch;
    struct {
        u<11> other0;
        unsigned other1:5;
    } __PACKED other;
} __PACKED;
```

Generated SystemVerilog:

```
package UnionWithStruct_pkg;

typedef union packed {
    struct packed {
        logic[24-1:0] _pad1;
        logic[5-1:0] other1;
        logic[11-1:0] other0;
    } other;
    struct packed {
        logic[5-1:0] _align0;
        logic[3-1:0] us1;
        UnionStruct nested;
        logic[6-1:0] _align1;
        logic[2-1:0] us0;
    } branch;
} UnionWithStruct;

endpackage
```

Templates

- During conversion, cpphdl uses numeric `Module` class template parameters as SystemVerilog module parameters. These numeric parameters stay symbolic in generated module ports, members, constants, and methods so one SV module can be instantiated with different numeric values.
- Static `constexpr` values declared by a module or its base classes are emitted as SystemVerilog `localparam` declarations inside the module. They may depend on numeric module parameters, but they are implementation constants and cannot be overridden by an instance.
- cpphdl creates a separate SV module for each instantiated combination of non-numeric `Module` template parameters. Type parameters and textual/string-like declaration parameters are specialization identity and are added to the generated module name.
- Template structs and unions are emitted only as concrete specializations. Numeric, type, and textual/string-like template arguments are all added to the generated package/type name, and an unspecialized template struct package must not be generated.
- Static `constexpr` values inside a template struct package are emitted as exact concrete values for that specialization. They must not reference unresolved template parameter names such as `WIDTH`, `CONV_TYPE`, or `TAG`.
- Static `constexpr` values inside a template module may reference numeric module parameters, but references that depend only on fixed type/text specializations should be resolved to concrete values.

References

- All references are removed during SV conversion. References should be used in C++ when necessary and when they improve performance.

Syntax

By default, a `generated` folder is created after a `cpphdl` call and contains the emitted `.sv` files. Use `--generated-dir <path>` to select another output directory. Converter options precede the source files; Clang compilation arguments such as defines and include directories follow them.

```
cpphdl [--generated-dir <path>] \
    [--primary_clock <name> <frequency>] \
    [--secondary_clock <name> <frequency>] ... \
    <source.h> <source.cpp> ... \
    [-DNAME=value] [-I<include-dir>] ...
```

The cpphdl tool is based on llvm clang and supports all usual C++ command line parameters.

Annotations

CPPHDL_REPLACEMENT

- `[[clang::annotate("CPPHDL_REPLACEMENT=...;")]]` can be attached to a `cpphdl::Module` class.
- `CPPHDL_REPLACEMENT_FILE=<path>;` reads the complete replacement from a file. Relative paths are resolved from the annotated class's source file when possible.
- `CPPHDL_REPLACEMENT_SCRIPT=<script> [arguments...];` executes a script and uses its standard output as the replacement. A relative script path is resolved in the same way as a replacement file.
- During conversion cpphdl resolves the inline text, file contents, or script output into `Module::replacement`; a trailing annotation metadata `;` is stripped from the annotation value.
- When project generation sees replacement text, it writes that text directly to the module `.sv` file.
- Normal import, port, register, method, and module body generation is skipped for that module.

Example from `tests/format/AnnotateReplacement.cpp`:

```
class [[clang::annotate(
    "CPPHDL_REPLACEMENT="
    "`default_nettype none\n"
    "\n"
    "module AnnotateReplacement (\n"
    "    input wire clk\n"
    ",   input wire reset\n"
    ",   input wire[8-1:0] value_in\n"
    ",   output wire[8-1:0] value_out\n"
    ");\n"
    "    assign value_out = value_in ^ 8'hA5;\n"
    "endmodule\n"
    ";"
)]] AnnotateReplacement : public Module
{
public:
    _PORT(u<8>) value_in;
    _PORT(u<8>) value_out = _ASSIGN_COMB(value_comb_func());

private:
    u<8> value_comb;

    u<8>& value_comb_func()
    {
        return value_comb = value_in() ^ u<8>(0xa5);
    }

public:
    void _work(bool reset) {}
    void _strobe() {}
    void _assign() {}
};
```